

Trabalho 3 e 4 - Encontrar o vetor θ^* e MRAC Direto

Alunos:

- Caio Cesar Leal Verissimo
- Leonardo Soares da Costa Tanaka
- Lincoln Rodrigues Proença

Professor:

- Ramon Romankevicius Costa

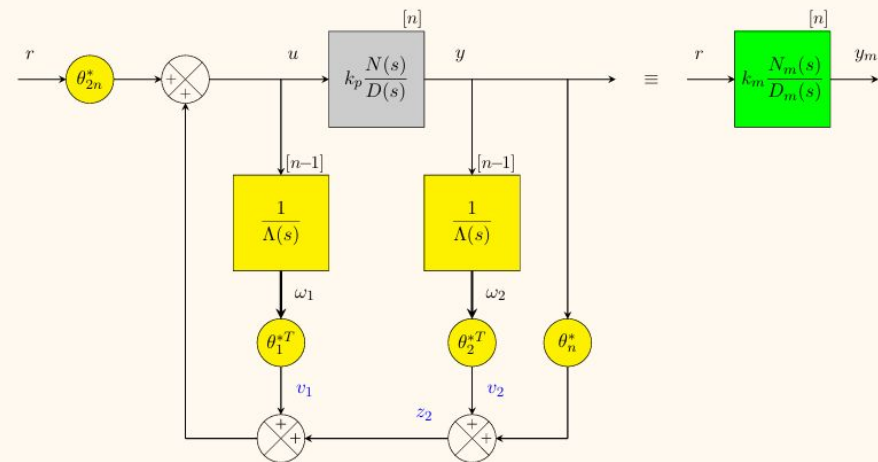
1. Introdução

Este trabalho tem como objetivo o desenvolvimento e simulação de algoritmos de **Controle Adaptativo Direto** aplicados a sistemas lineares. Na primeira etapa, é proposta a implementação de uma função em MATLAB capaz de determinar os parâmetros ideais θ^* para controle 2DOF, considerando diferentes ordens de planta, modelos de referência e polinômios do observador. Na segunda etapa, realiza-se a implementação e análise do algoritmo **MRAC Direto**, tanto para grau relativo $n^*=1$ quanto $n^*=2$, avaliando seu desempenho frente a diferentes condições iniciais, ganhos de adaptação e sinais de referência. As atividades visam consolidar os conceitos teóricos de controle adaptativo, com foco na análise de estabilidade e desempenho dos controladores.



2. Problema de controle 2DOF

2.1. Estrutura do MRAC



Podemos verificar que esta estrutura é equivalente à 2DOF.

Filtro 1:
$$\begin{cases} \dot{\omega}_1 = A_f \omega_1 + b_f u \\ v_1 = \theta_1^{*T} \omega_1 \end{cases} \Rightarrow \boxed{v_1 = \frac{F}{\Lambda} u}$$

- $\boxed{\text{grau}(F) = n - 2}$

★ Note que **não há ramo direto** no filtro de u (geraria um loop algébrico).

Filtro 2:
$$\begin{cases} \dot{\omega}_2 = A_f \omega_2 + b_f y \\ z_2 = \theta_2^{*T} \omega_2 + \theta_n^* y \end{cases} \Rightarrow \boxed{z_2 = \frac{G}{\Lambda} y}$$

- $\boxed{\text{grau}(G) = n - 1}$

★ Note que há um ramo direto.

Lei de controle:
$$\boxed{u = \theta^{*T} \omega}$$

Parâmetros:
$$\boxed{\theta^{*T} = [\theta_1^{*T} \quad \theta_n^* \quad \theta_2^{*T} \quad \theta_{2n}^*]} \quad \theta^* \in \mathbb{R}^{2n}$$

Regressor:
$$\boxed{\omega^T = [\omega_1^T \quad y \quad \omega_2^T \quad r]} \quad \omega \in \mathbb{R}^{2n}$$

2.2. Malha Fechada

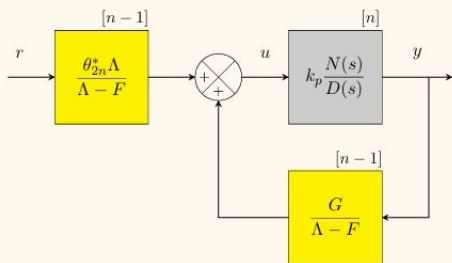


Figura 58: Estrutura do MRAC.

Malha fechada:
$$y = k_p \frac{N}{D} \left(\frac{\theta_{2n}^* \Lambda}{\Lambda - F} r + \frac{G}{\Lambda - F} y \right) \Rightarrow$$

$$\Rightarrow D(\Lambda - F)y = \theta_{2n}^* k_p \Lambda N r + k_p N G y \Rightarrow \boxed{y = \frac{\theta_{2n}^* k_p \Lambda N}{(\Lambda - F)D - k_p N G} r}$$

Matching antes dos cancelamentos:

$$\boxed{\frac{\theta_{2n}^* k_p \Lambda N}{(\Lambda - F)D - k_p N G} = \frac{k_m N_m N A_0}{D_m N A_0}}$$

★ A_0 : polinômio do observador.

★ Cancelamentos necessários: N e A_0 .

Condições necessárias para a solução:

$$\bullet \boxed{\theta_{2n}^* = \frac{k_m}{k_p}} \Rightarrow \theta_{2n}^* k_p = k_m$$

$$\bullet \boxed{\Lambda = N_m A_0} \quad (\text{Para o matching do numerador.})$$

$$\bullet \begin{cases} \text{grau}(\Lambda) = n - 1 \\ \text{grau}(N_m) = m \end{cases} \Rightarrow \boxed{\text{grau}(A_0) = n - m - 1 = n^* - 1}$$

Para o cancelamento de N é necessário que

- $(\Lambda - F) = NH$ (N deve ser fator de $\Lambda - F$)
- $\begin{cases} \text{grau}(\Lambda) = n - 1 \\ \text{grau}(F) = n - 2 \\ \text{grau}(N) = m \end{cases} \Rightarrow \text{grau}(H) = n - m - 1 = n^* - 1$

Hipótese: $P(s)$ é controlável e observável.

★ $N(s)$ e $D(s)$ são primos!

Após o cancelamento de N , tem-se a equação

$$HD - k_p G = D_m A_0 \quad (\text{Equação Diophantina.})$$

- $\text{grau}(HD) = \text{grau}(H) + \text{grau}(D) = (n - m - 1) + n = 2n - m - 1$
- $\text{grau}(G) = n - 1$
- $\text{grau}(D_m A_0) = \text{grau}(D_m) + \text{grau}(A_0) = n + (n - m - 1) = 2n - m - 1$

★ $H(s)$ é **mônico**.

Hipótese: Grau relativo de $P(s)$ = Grau relativo de $M(s)$.

$$\text{grau}(D) - \text{grau}(N) = \text{grau}(D_m) - \text{grau}(N_m) = n^*$$

★ O grau relativo é invariante.

2.3. Resumo do Algoritmo

(1) Determinar $\text{grau}(A_0)$ e $\text{grau}(G)$.

(2) Achar a solução $H(s)$ e $G(s)$ da equação *Diophantina*

$$H(s)D(s) - k_p G(s) = D_m(s)A_0(s)$$

(3) Obter θ_2^* e θ_n^* a partir de $G(s)$.

(4) Calcular $F(s)$ usando a relação

$$F = \Lambda(s) - N(s)H(s)$$

(5) Obter θ_1^* a partir de $F(s)$.

$$(6) \theta_{2n}^* = \frac{k_m}{k_p}.$$

2.4. Implementação do algoritmo

```
function [theta1, theta_n, theta2, theta_2n, L] = controle2DOF(P, M, A0)
```

```
% =====  
% Função para cálculo dos vetores  $\theta^*$  (controle 2DOF)  
% Entrada:  
% - P : função de transferência da planta  
% - M : função de transferência do modelo de referência  
% - A0: polinômio característico do observador (como vetor)  
% Saída:  
% - theta1 : vetor do filtro na entrada  
% - theta_n: ganho direto na saída  
% - theta2 : vetor do filtro na saída  
% - theta_2n: ganho no feedforward (referência)  
% - L: Filtro Lambda  
% =====
```

```
syms s
```

```
% =====  
% Obter dados  
% =====
```

```
[N_p, D_p] = tfdata(P, 'v');  
[N_m, D_m] = tfdata(M, 'v');
```

```
% =====  
% Normalização  
% =====
```

```
kp = N_p(find(N_p ~= 0, 1, 'first'));  
km = N_m(find(N_m ~= 0, 1, 'first'));  
Np = poly2sym(N_p, s)/kp;  
Dp = poly2sym(D_p, s);  
Nm = poly2sym(N_m, s)/km;  
Dm = poly2sym(D_m, s);  
A0_sym = poly2sym(A0, s);
```

```
% =====  
% Graus dos polinômios  
% =====
```

```
n = length(D_p) - 1;  
m = length(N_p(find(N_p~=0, 1, 'first'):end)) - 1;  
n_star = n - m;
```



```

%% =====
% Cálculo de  $\Lambda(s)$ 
% =====

Lambda = Nm * A0_sym;

% Verificar grau de Lambda
Lambda_coeffs = sym2poly(expand(Lambda));
grau_Lambda = length(Lambda_coeffs) - 1;

% Se grau de Lambda for menor que n - 1, multiplicar por (s +  $\lambda_i$ ) até atingir o grau desejado
k = 1; % índice para gerar lambdas distintos
while grau_Lambda < n - 1
    %lambda_k = sym(['lambda', num2str(k)]);
    lambda_k = 2*k;
    fator = s + lambda_k;
    Nm = expand(Nm * fator);
    Dm = expand(Dm * fator);
    Lambda = Nm * A0_sym;
    grau_Lambda = grau_Lambda + 1;
    k = k + 1;
end

%% =====
% Resolver equação Diofantina:
%  $H(s)*D_p + G(s) = Dm * A0$ 
% =====
deg_H = n_star - 1;
deg_G = n - 1;

H = sym('h', [1 deg_H]);
G = sym('g', [1 deg_G + 1]);

H_poly = poly2sym([1 H], s);
G_poly = poly2sym(G, s);

% Montagem da equação Diofantina
Eq = expand(H_poly * Dp - kp*G_poly - Dm * A0_sym);

% Coletar os coeficientes em potências de s
coeffs_Eq = fliplr(coeffs(Eq, s));

% Sistema de equações
eqns = coeffs_Eq == zeros(1, length(coeffs_Eq));

% Resolver equações

vars = [H G];
sol = solve(eqns, vars);

% Obter nomes dos campos da solução
fields = fieldnames(sol);

% Processar H
h_fields = fields(startsWith(fields, 'h'));

if isempty(h_fields)
    H_sol = []; % Caso não exista H (grau zero)
    H_final = 1; %  $H(s) = 1$ 
else
    h_fields = sort(h_fields); % Ordena os nomes dos coeficientes
    H_cells = cellfun(@(c) sol.(c), h_fields, 'UniformOutput', false);
    H_sol = [H_cells{:}]; % Coeficientes do menor para o maior grau ( $h_1, \dots, h_n$ )

    grau_H = length(H_sol);
    H_final = s^(grau_H); % Começa com  $s^n$ 

    for i = 1:grau_H
        H_final = H_final + H_sol(i) * s^(grau_H - i);
    end

    H_final = expand(H_final); % Expande a expressão simbólica
end

```

```

%% Processar G
g_fields = fields(startsWith(fields, 'g'));

if isempty(g_fields)
    G_sol = []; % Caso não exista G
    G_final = 0; % Polinômio nulo
else
    g_fields = sort(g_fields); % Ordena lexicograficamente
    G_cells = cellfun(@(c) sol.(c), g_fields, 'UniformOutput', false);
    G_sol = [G_cells{:}]; % Vetor linha com os coeficientes

    % Construir  $G(s) = G_{sol}(1)*s^n + G_{sol}(2)*s^{(n-1)} + \dots + G_{sol}(\text{end})$ 
    grau_G = length(G_sol) - 1;
    G_final = 0;
    for i = 0:grau_G
        G_final = G_final + G_sol(i+1) * s^(grau_G - i);
    end
    G_final = expand(G_final);
end

```

```

%% =====
% Cálculo de F(s)
%  $F = \Lambda - N_p * H$ 
% =====
F = expand(Lambda - Np * H_final);

%% Cálculo dos thetas
theta_2n = km / kp;

% Representações simbólicas
G_coeffs = G_cells;
G_coeffs = G_coeffs(:).'; % garantir vetor linha

[Lambda_coeffs, ~] = coeffs(Lambda, s);
Lambda_coeffs = Lambda_coeffs(:).';

% Graus
grau_G = length(G_coeffs) - 1;
grau_L = length(Lambda_coeffs) - 1;
len_theta = n - 1;

% Variáveis simbólicas
theta_n_sym = sym('theta_n_scalar');
theta2_sym = sym('t', [1 len_theta]);

%  $\theta_2(s) + \theta_n * \Lambda(s)$ 
theta2_poly = poly2sym(theta2_sym, s);
theta_n_term = theta_n_sym * Lambda;
G_est = expand(theta2_poly + theta_n_term);

```

```

% Igualar coeficientes
G_est_coeffs = fliplr(coeffs(G_est, s));
G_coeffs_target = [zeros(1, length(G_est_coeffs) - length(G_coeffs)), G_coeffs];

% Resolver sistema
eqns = G_est_coeffs == G_coeffs_target;
vars = [theta2_sym, theta_n_sym];
sol = solve(eqns, vars);

% Extrair soluções
theta2 = double(arrayfun(@(i) sol.(sprintf('t%d', i)), 1:len_theta));
theta_n = double(sol.theta_n_scalar);

%  $\theta_1 \rightarrow$  Coeficientes de F(s) normalizados por  $\Lambda$ 
thetal = sym2poly(F);
thetal = double(thetal);

L = double(fliplr(coeffs(Lambda, s)));

%% Exibir resultados
fprintf('\n=== Resultados ===\n');
fprintf('theta_1 = %s\n', num2str(thetal));
fprintf('theta_n = %s\n', num2str(theta_n));
fprintf('theta_2 = %s\n', num2str(theta2));
fprintf('theta_2n = %s\n', num2str(theta_2n));
fprintf('Lambda = %s\n', Lambda);

```

2.5.1. Resultado da simulação ($n^*=1$)

Classificação do sistema: $n = 2$ (ordem)
 $n^* = 1$ (grau relativo)
 $n_p = 4$ (# de parâmetros)

Planta.....: $y = \frac{k_p(s+b)}{s^2 + a_1s + a_0}u$

Modelo.....: $y_m = \frac{k_m(s+b_m)}{s^2 + a_{1m}s + a_{0m}}r$

$$\theta_1^* = b_m - b$$

$$\theta_n^* = \frac{a_1 - a_{1m}}{k_p}$$

$$\theta_2^* = \frac{(a_0 - a_{0m}) - b_m(a_1 - a_{1m})}{k_p}$$

$$\theta_{2n}^* = \frac{k_m}{k_p}$$

```
%% Definir Planta
```

```
kp = 0.1;
```

```
b = 3;
```

```
P = tf(kp*[1 b], [1 5 3]);
```

```
%% Definir Modelo
```

```
km = 1;
```

```
bm = 2;
```

```
M = tf(km*[1 bm], [1 2 1]);
```

```
%% Polinômio do Observador
```

```
A0 = 1;
```

```
=== Resultados ===
```

```
theta_1 = -1
```

```
theta_n = 30
```

```
theta_2 = -40
```

```
theta_2n = 10
```



2.5.2. Resultado da simulação ($n^*=2$)

Classificação do sistema: $n = 2$ (ordem)
 $n^* = 2$ (grau relativo)
 $n_p = 4$ (# de parâmetros)

Planta.....: $y = \frac{k_p}{s^2 + a_1s + a_0}u$

Modelo.....: $y_m = \frac{k_m}{s^2 + a_{1m}s + a_{0m}}r$

$$\theta_1^* = a_1 - a_{1m}$$

$$\theta_n^* = \frac{(a_0 - a_{0m}) + (\lambda_0 - a_1)(a_1 - a_{1m})}{k_p}$$

$$\theta_2^* = \frac{-(a_0 + \lambda_0(\lambda_0 - a_1))(a_1 - a_{1m})}{k_p}$$

$$\theta_{2n}^* = \frac{k_m}{k_p}$$

```
%% Definir Planta
```

```
kp = 2;
```

```
P = tf(kp, [1 4 3]);
```

```
%% Definir Modelo
```

```
km = 1;
```

```
M = tf(km, [1 5 6]);
```

```
%% Polinômio do Observador
```

```
A0 = [1 10];
```

```
=== Resultados ===
```

```
theta_1 = -1
```

```
theta_n = -4.5
```

```
theta_2 = 31.5
```

```
theta_2n = 0.5
```





3. MRAC Direto ($n^*=1$)



3.1. Algoritmo ($n^*=1$)

Subsistema	Equação	Ordem
Planta	$y = P(s) u$	n
Modelo	$y_m = M(s) r$	n
Erro	$e_a = \text{sign}(k_p)(y - y_m)$	
Controle	$u = \theta^T \omega$	
Filtros	$\dot{\omega}_1 = A_f \omega_1 + b_f u$	$n - 1$
	$\dot{\omega}_2 = A_f \omega_2 + b_f y$	$n - 1$
Regressor	$\omega^T = [\omega_1^T \quad y \quad \omega_2^T \quad r]$	
Adaptação	$\dot{\theta} = -\Gamma \omega e_a$	$2n$

3.2. Critério de Estabilidade

Control law:

$$u = \theta^T \omega$$

Set of equations:

$$\begin{cases} \dot{e} = Ae + k_p B[\tilde{\theta}^T \omega], & e_0 = Ce \\ \dot{\theta} = ? \end{cases}$$

Lyapunov function:

$$2V(e, \tilde{\theta}) = e^T P e + |k_p| \tilde{\theta}^T \Gamma^{-1} \tilde{\theta} \quad \Gamma = \Gamma^T > 0$$

Time derivative:

$$\begin{aligned} 2\dot{V} &= \dot{e}^T P e + e^T P \dot{e} + |k_p| (\dot{\theta}^T \Gamma^{-1} \tilde{\theta} + \tilde{\theta}^T \Gamma^{-1} \dot{\theta}) \\ &= (Ae + k_p B \tilde{\theta}^T \omega)^T P e + e^T P (Ae + k_p B \tilde{\theta}^T \omega) + 2|k_p| \tilde{\theta}^T \Gamma^{-1} \dot{\theta} \\ &= e^T A^T P e + k_p \tilde{\theta}^T \omega B^T P e + e^T P A e + k_p e^T P B \tilde{\theta}^T \omega + 2|k_p| \tilde{\theta}^T \Gamma^{-1} \dot{\theta} \\ &= e^T (\underbrace{A^T P + P A}_{-2Q}) e + 2k_p \tilde{\theta}^T \omega B^T P e + 2|k_p| \tilde{\theta}^T \Gamma^{-1} \dot{\theta} \\ &= -2e^T Q e + 2k_p \tilde{\theta}^T \omega B^T P e + 2|k_p| \tilde{\theta}^T \Gamma^{-1} \dot{\theta} \end{aligned}$$

Therefore,

$$\begin{aligned} \dot{V} &= -e^T Q e + k_p \tilde{\theta}^T \omega B^T P e + |k_p| \tilde{\theta}^T \Gamma^{-1} \dot{\theta} \\ &= -e^T Q e + |k_p| \tilde{\theta}^T \left[\text{sign}(k_p) \omega \underbrace{B^T P e}_{Ce=e_0} + \Gamma^{-1} \dot{\theta} \right] \end{aligned}$$

We choose the update law:

$$\dot{\theta} = -\text{sign}(k_p) \Gamma \omega e_0$$

Result:

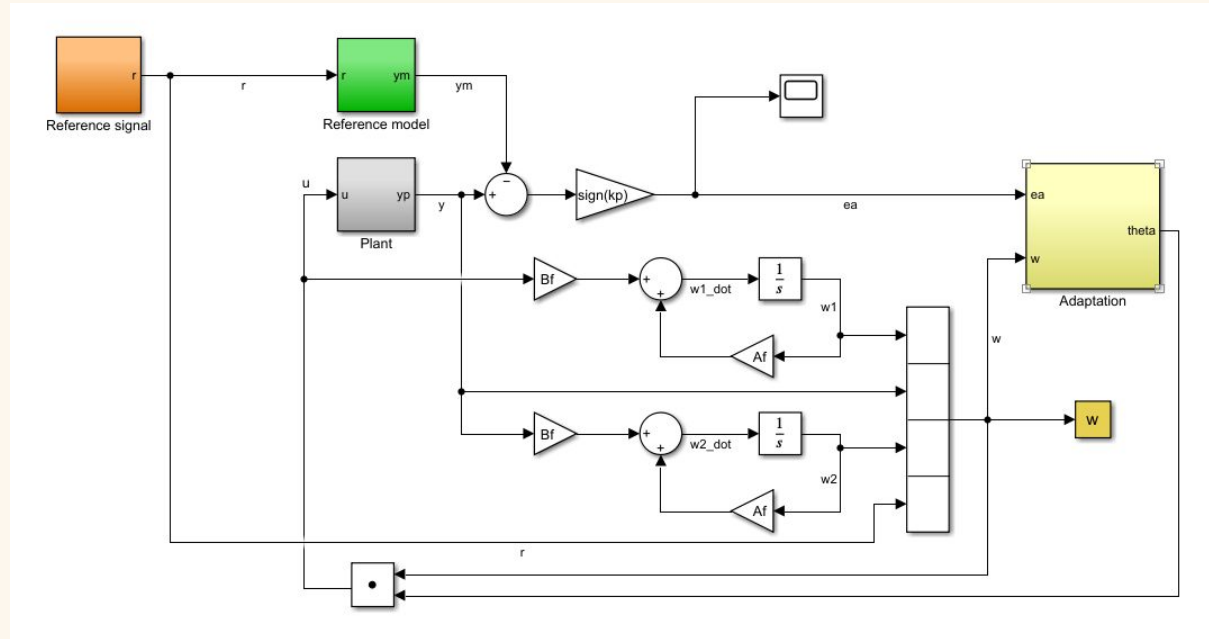
$$\dot{V} = -e^T Q e \leq 0$$

- $V(t)$ is monotone non-increasing, bounded above by $V(0)$ and below by 0.
- $e, \tilde{\theta} \in \mathcal{L}_\infty$
- Integrating $\dot{V} \Rightarrow e \in \mathcal{L}_2$
- $r(t) \in \mathcal{L}_\infty \Rightarrow \dot{e} \in \mathcal{L}_\infty$
- $\begin{cases} e \in \mathcal{L}_2 \\ \dot{e} \in \mathcal{L}_\infty \end{cases} \Rightarrow e \rightarrow 0$
- Convergence of $\tilde{\theta} \rightarrow 0$ requires the **persistent excitation** condition

$$\int_t^{t+T} \omega(\tau) \omega^T(\tau) d\tau \geq \alpha I \quad T, \alpha > 0, \quad \forall t \geq t_0$$

★ Note that $\omega(\cdot)$ is an internal signal of the system.

3.3. Diagramas de Blocos





3.4. Resultados das simulações



3.4.1. Base

P =

$$\frac{2.916 s^2 + 11.66 s + 8.747}{s^3 + 9 s^2 + 20 s + 12}$$

Continuous-time transfer function.

ans =

-6.0000
-2.0000
-1.0000

ans =

-3
-1

=== Resultados ===

theta_1 = 2 5
theta_n = 0
theta_2 = -1.0289 -1.0289
theta_2n = 0.64306
Lambda = 6*s + s^2 + 8

M =

$$\frac{1.875 s^2 + 11.25 s + 15}{s^3 + 9 s^2 + 23 s + 15}$$

Continuous-time transfer function.

ans =

-5.0000
-3.0000
-1.0000

ans =

-4
-2

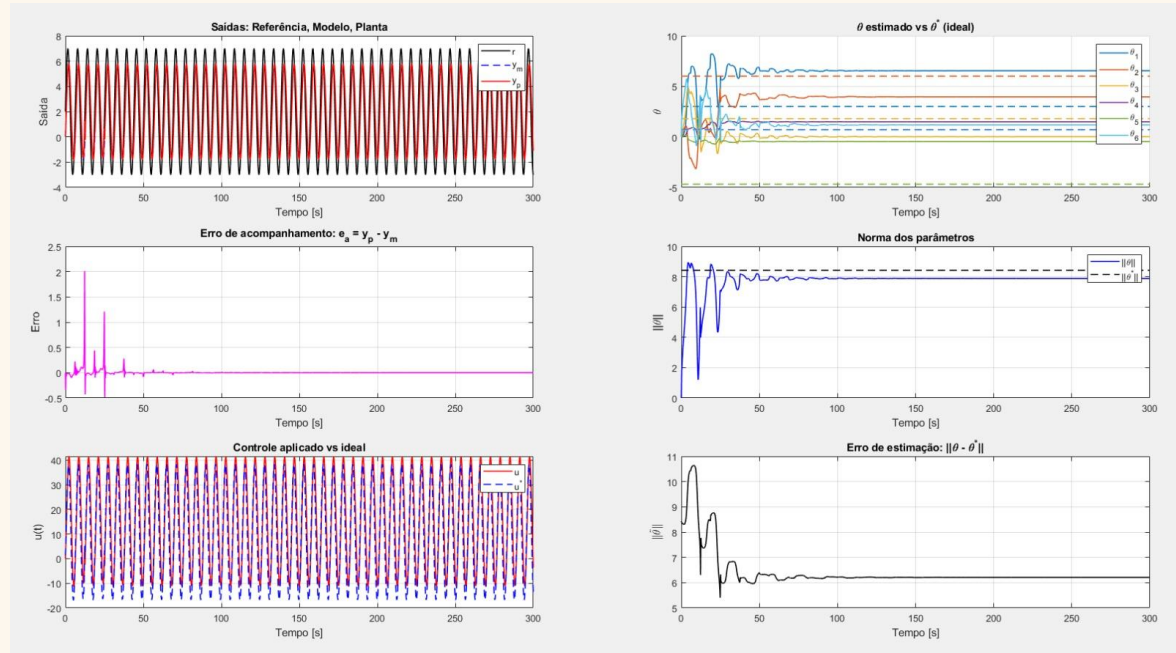
3.4.1. Base

```
gamma = 10*eye(2*n);  
tfinal = 300;  
st = 0.01;  
theta0 = zeros(2*n, 1);
```

```
DC = 2    %Constant  
Aq = 0    %Sqr wave amplitude  
wq = 0.1*pi %Frequency  
As = 5    %Sine wave amplitude  
ws = 1%pi %Frequency
```

```
x0 = zeros(n, 1);
```

```
A0 = 1;
```



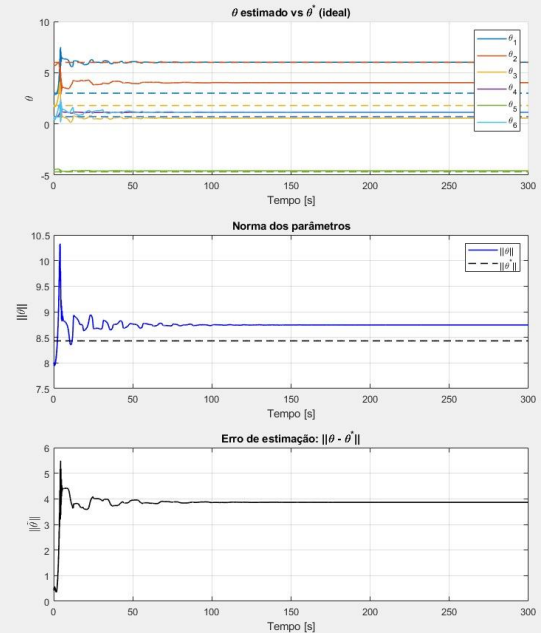
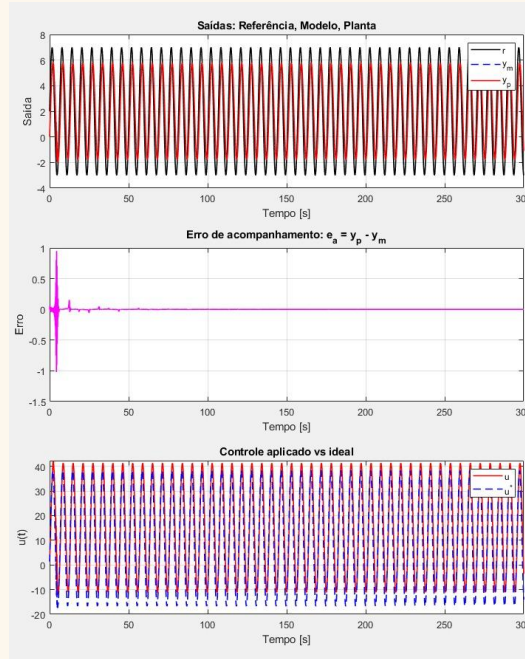
3.4.2. $0.95 \theta^*$

```
gamma = 10*eye(2*n);  
tfinal = 300;  
st = 0.01;  
theta0 = 0.95*theta_star;
```

```
DC = 2    %Constant  
Aq = 0    %Sqr wave amplitude  
wq = 0.1*pi %Frequency  
As = 5    %Sine wave amplitude  
ws = 1%pi %Frequency
```

```
x0 = zeros(n, 1);
```

```
A0 = 1;
```



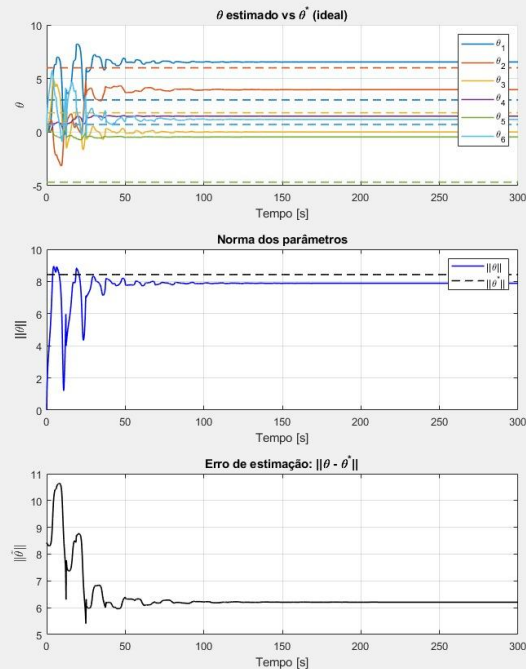
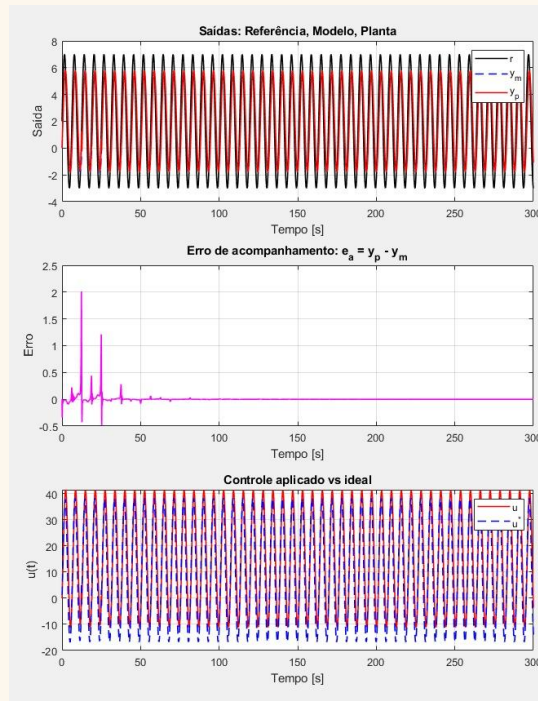
3.4.3. Condições Iniciais

```
gamma = 10*eye(2*n);  
tfinal = 300;  
st = 0.01;  
theta0 = zeros(2*n, 1);
```

```
DC = 2 %Constant  
Aq = 0 %Sqr wave amplitude  
wq = 0.1*pi %Frequency  
As = 5 %Sine wave amplitude  
ws = 1%pi %Frequency
```

```
x0 = ones(n, 1);
```

```
A0 = 1;
```



3.4.4. Ganho de adaptação

```
gamma = 100*eye(2*n);
```

```
tfinal = 300;
```

```
st = 0.01;
```

```
theta0 = zeros(2*n, 1);
```

```
DC = 2 %Constant
```

```
Aq = 0 %Sqr wave amplitude
```

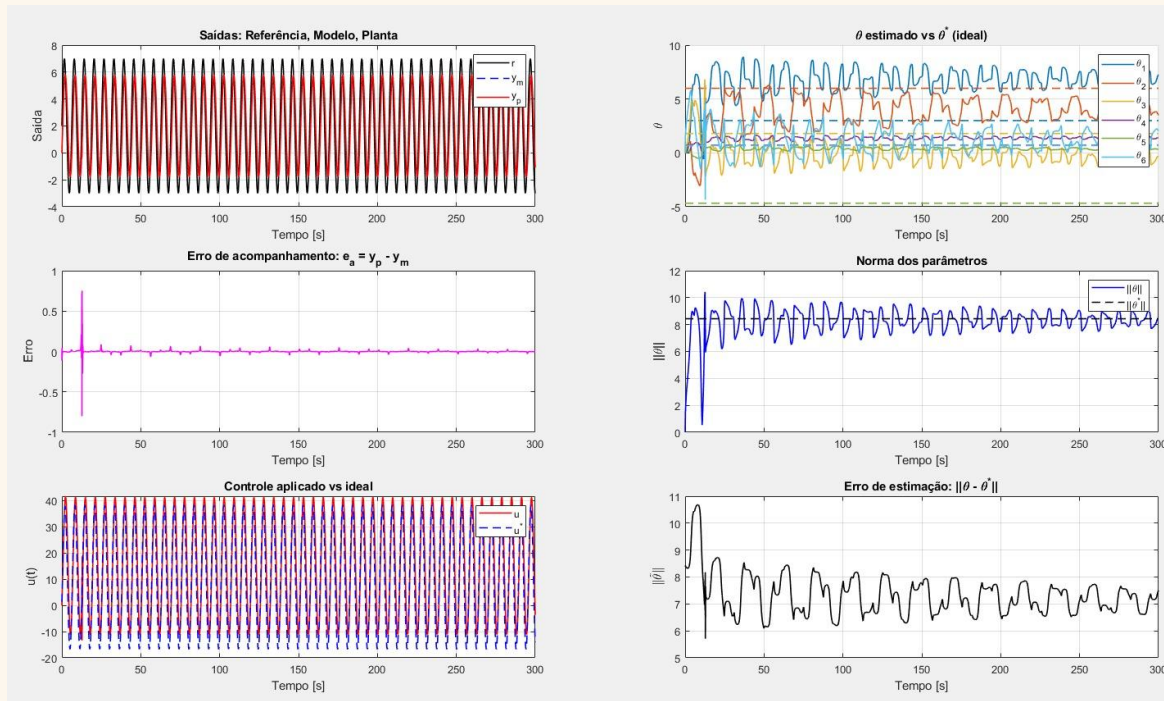
```
wq = 0.1*pi %Frequency
```

```
As = 5 %Sine wave amplitude
```

```
ws = 1%pi %Frequency
```

```
x0 = zeros(n, 1);
```

```
A0 = 1;
```



3.4.5. Sinal de Referência

```
gamma = 100*eye(2*n);  
tfinal = 300;  
st = 0.01;  
theta0 = zeros(2*n, 1);
```

```
DC = 2 %Constant
```

```
Aq = 2 %Sqr wave amplitude
```

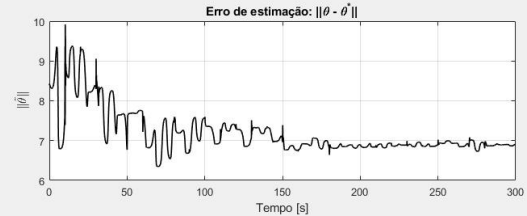
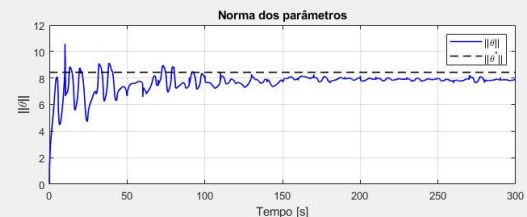
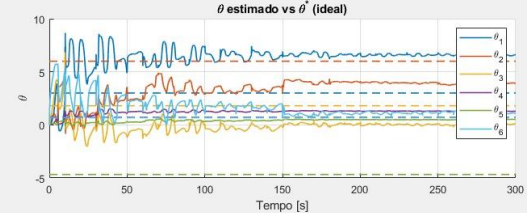
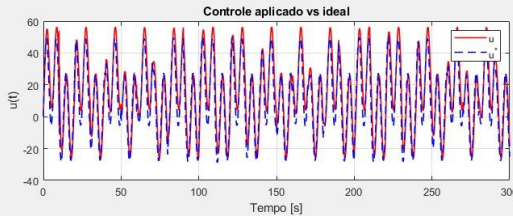
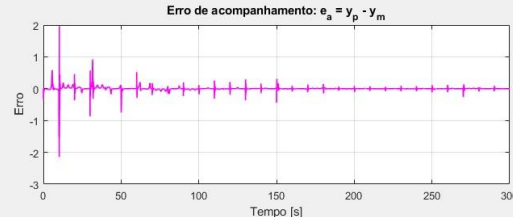
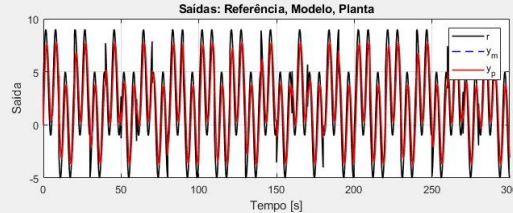
```
wq = 0.1*pi %Frequency
```

```
As = 5 %Sine wave amplitude
```

```
ws = 1%pi %Frequency
```

```
x0 = zeros(n, 1);
```

```
A0 = 1;
```



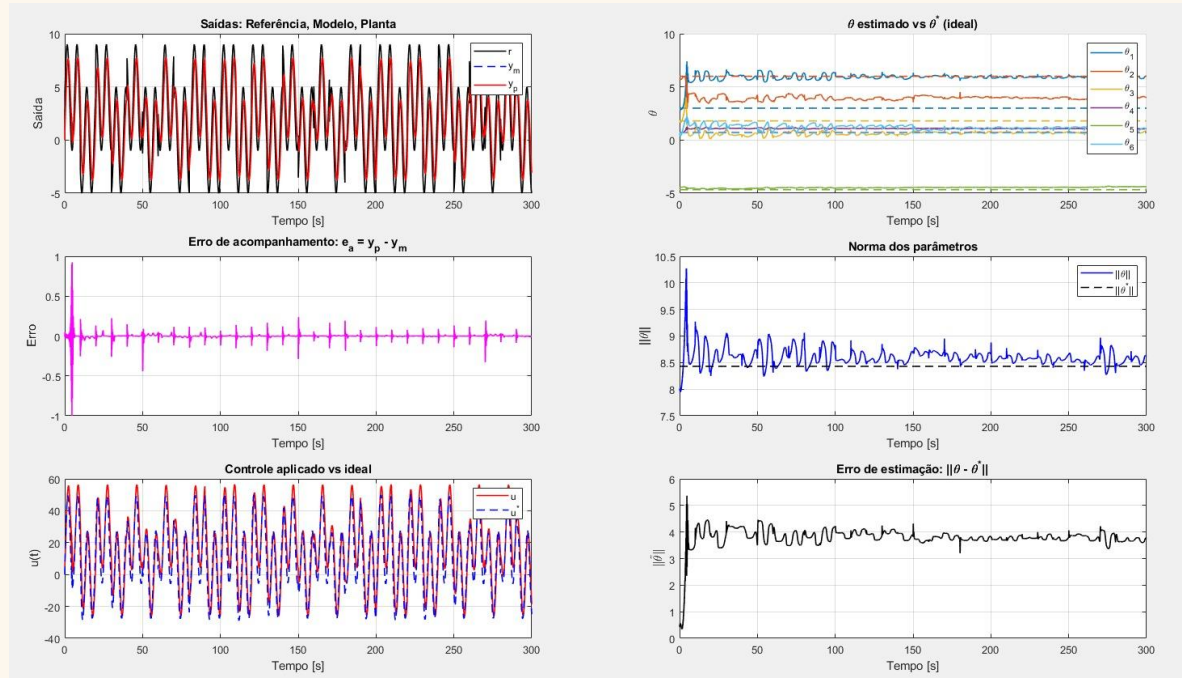
3.4.6. Sinal de Referência com $0.95 \theta^*$

```
gamma = 100*eye(2*n);  
tfinal = 300;  
st = 0.01;  
theta0 = 0.95*theta_star;
```

```
DC = 2 %Constant  
Aq = 2 %Sqr wave amplitude  
wq = 0.1*pi %Frequency  
As = 5 %Sine wave amplitude  
ws = 1*pi %Frequency
```

```
x0 = zeros(n, 1);
```

```
A0 = 1;
```





4. MRAC Direto ($n^*=2$)



4.1. Algoritmo ($n^*=2$)

Subsystem	Equation	Dimension
Plant	$y = P(s) u$	n
Model	$y_m = M(s) r$	n
Tracking error	$e_a = \text{sign}(k_p) (y - y_m)$	
Control law	$u = \theta^T \omega + \dot{\theta}^T \xi$	
SV filters	$\dot{\omega}_1 = A_f \omega_1 + b_f u$	$n - 1$
	$\dot{\omega}_2 = A_f \omega_2 + b_f y$	$n - 1$
Regressor	$\omega^T = [\omega_1^T \quad y \quad \omega_2^T \quad r]$	
ξ -Filter	$\dot{\xi} = -\ell_0 \xi + \omega, \quad \ell_0 > 0$	$2n$
Update law	$\dot{\theta} = -\Gamma \xi e_a$	$2n$

4.2. Critério de Estabilidade

The derivation of the update law is similar to the previous case.

Error equation: $\Rightarrow \boxed{e_a = \underbrace{|k_p| ML}_{\text{SPR}} [\tilde{\theta}^T \xi]}$

State space representation:
$$\begin{cases} \dot{e} = A_m e + B_m [\tilde{\theta}^T \xi] \\ e_a = C_m e \end{cases}$$

★ $\{A_m, B_m, C_m\}$ is a **non-minimal realization** of $|k_p| M(s)L(s)$.

★ The error vector $e \in \mathbb{R}^{5n-2}$.

Since the system $\{A_m, B_m, C_m\}$ is SPR then

$$\exists \begin{cases} P = P^T > 0 \\ Q = Q^T > 0 \end{cases} \quad \text{s.t.} \quad \begin{cases} A_m^T P + P A_m = -2Q \\ P B_m = C_m^T \end{cases}$$

Set of equations:
$$\begin{cases} \dot{e} = A_m e + B_m [\tilde{\theta}^T \xi], & e_a = C_m e \\ \dot{\theta} = ? \end{cases}$$

Now, consider the Lyapunov function

$$\boxed{2V(e, \tilde{\theta}) = e^T P e + \tilde{\theta}^T \Gamma^{-1} \tilde{\theta}} \quad \Gamma = \Gamma^T > 0$$

Time derivative:

$$\begin{aligned} 2\dot{V} &= \dot{e}^T P e + e^T P \dot{e} + (\dot{\theta}^T \Gamma^{-1} \tilde{\theta} + \tilde{\theta}^T \Gamma^{-1} \dot{\theta}) \\ &= (A_m e + B_m \tilde{\theta}^T \xi)^T P e + e^T P (A_m e + B_m \tilde{\theta}^T \xi) + 2\tilde{\theta}^T \Gamma^{-1} \dot{\theta} \\ &= e^T A_m^T P e + \tilde{\theta}^T \xi B_m^T P e + e^T P A_m e + e^T P B_m \tilde{\theta}^T \xi + 2\tilde{\theta}^T \Gamma^{-1} \dot{\theta} \\ &= e^T \underbrace{(A_m^T P + P A_m)}_{-2Q} e + 2\tilde{\theta}^T \xi B_m^T P e + 2\tilde{\theta}^T \Gamma^{-1} \dot{\theta} \\ &= -2e^T Q e + 2\tilde{\theta}^T \xi B_m^T P e + 2\tilde{\theta}^T \Gamma^{-1} \dot{\theta} \end{aligned}$$

Therefore,

$$\begin{aligned} \dot{V} &= -e^T Q e + \tilde{\theta}^T \xi B_m^T P e + \tilde{\theta}^T \Gamma^{-1} \dot{\theta} \\ &= -e^T Q e + \tilde{\theta}^T \Gamma^{-1} \underbrace{[\Gamma \xi B_m^T P e + \dot{\theta}]}_{C_m e = e_a} \end{aligned}$$

We choose the update law:

$$\boxed{\dot{\theta} = -\Gamma \xi e_a}$$

Result:

$$\boxed{\dot{V} = -e^T Q e \leq 0}$$

★ The previous regressor ω has been replaced by ξ .

★ The dynamics of the filter $L^{-1}(s)$ has been included.

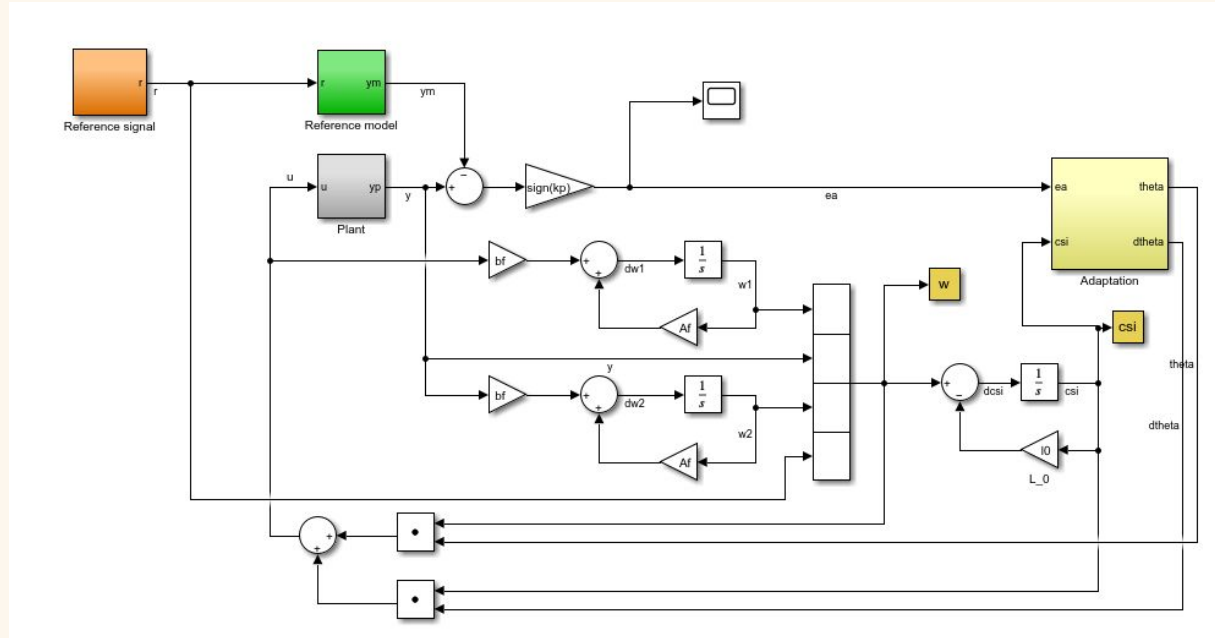
From the above analysis, we have that

$$\boxed{\omega \in \mathcal{L}_\infty}.$$

Since $L^{-1}(s)$ is **BIBO**, then

$$\xi = \underbrace{L^{-1}(s)}_{\text{BIBO}} \underbrace{\omega}_{\mathcal{L}_\infty} \Rightarrow \boxed{\xi \in \mathcal{L}_\infty}$$

4.3. Diagramas de Blocos





4.4. Resultados das simulações



4.4.1. Base

P =

$$\frac{2.351 s + 9.404}{s^3 + 16 s^2 + 73 s + 90}$$

Continuous-time transfer function.

ans =

-9.0000
-5.0000
-2.0000

ans =

-4

=== Resultados ===

theta_1 = 5 8
theta_n = 3.4029
theta_2 = -13.6115 -17.0144
theta_2n = 3.1902
Lambda = (s + 2)*(s + 10)

M =

$$\frac{7.5 s + 15}{s^3 + 9 s^2 + 23 s + 15}$$

Continuous-time transfer function.

ans =

-5.0000
-3.0000
-1.0000

ans =

-2

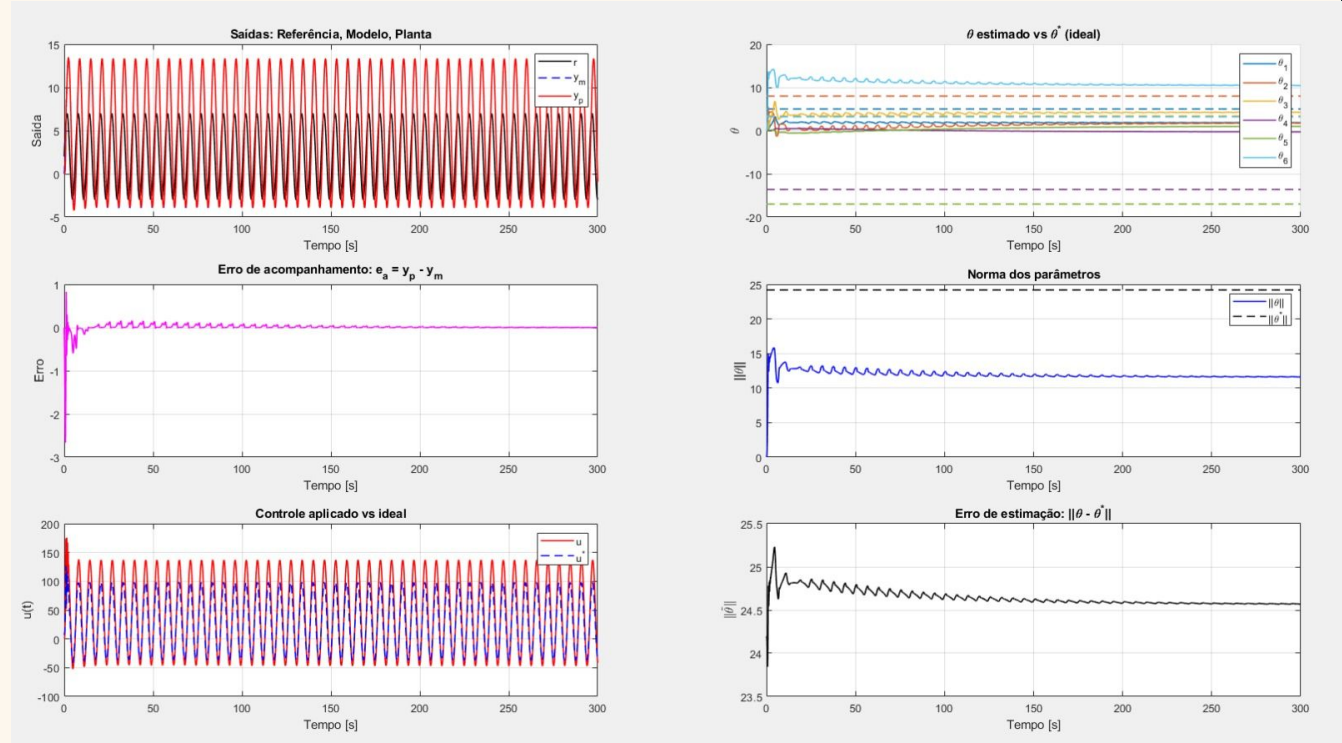
4.4.1. Base

```
gamma = 10*eye(2*n);  
tfinal = 300;  
st = 0.01;  
theta0 = zeros(2*n, 1);
```

```
DC = 2 %Constant  
Aq = 0 %Sqr wave amplitude  
wq = 0.1*pi %Frequency  
As = 5 %Sine wave amplitude  
ws = 1%pi %Frequency
```

```
x0 = zeros(n, 1);
```

```
A0 = [1 10];
```



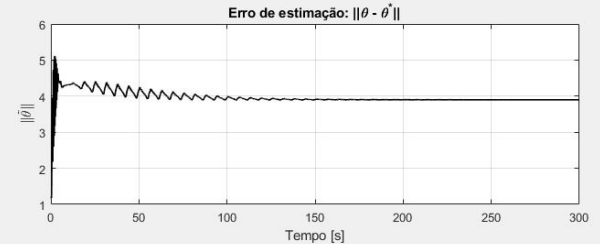
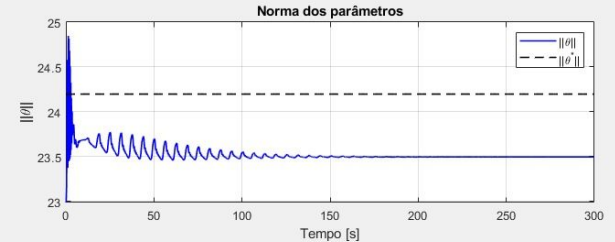
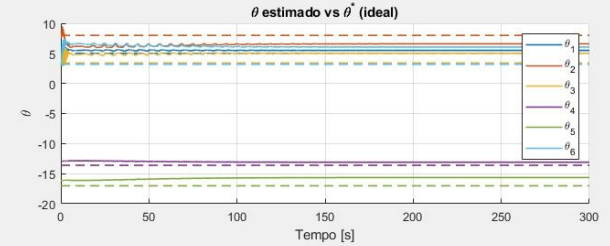
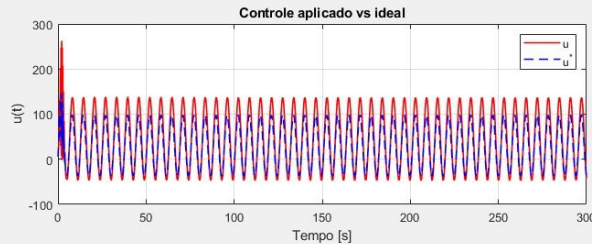
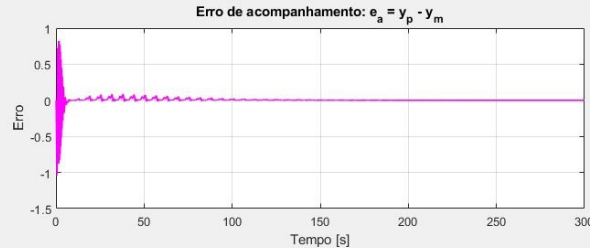
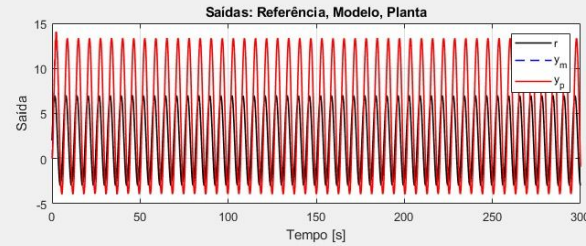
4.4.2. $0.95 \theta^*$

```
gamma = 10*eye(2*n);  
tfinal = 300;  
st = 0.01;  
theta0 = 0.95*theta_star;
```

```
DC = 2 %Constant  
Aq = 0 %Sqr wave amplitude  
wq = 0.1*pi %Frequency  
As = 5 %Sine wave amplitude  
ws = 1%pi %Frequency
```

```
x0 = zeros(n, 1);
```

```
A0 = [1 10];
```



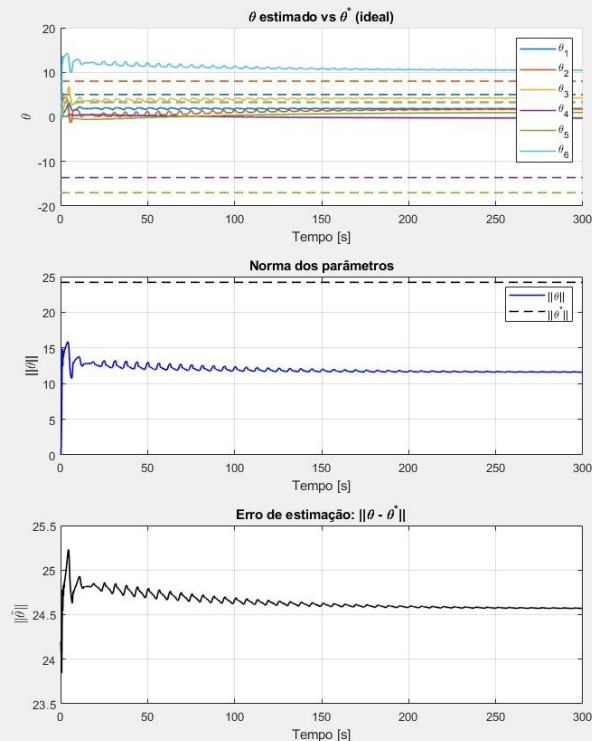
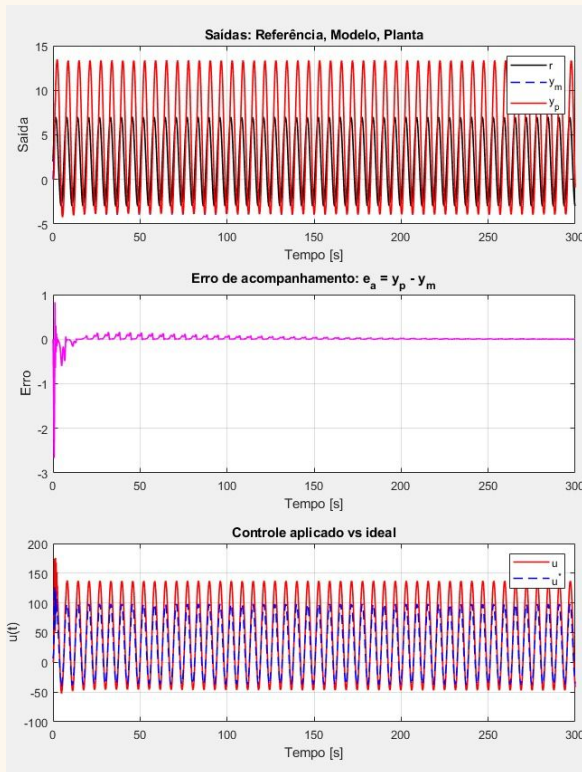
4.4.3. Condições Iniciais

```
gamma = 10*eye(2*n);  
tfinal = 300;  
st = 0.01;  
theta0 = zeros(2*n, 1);
```

```
DC = 2 %Constant  
Aq = 0 %Sqr wave amplitude  
wq = 0.1*pi %Frequency  
As = 5 %Sine wave amplitude  
ws = 1%pi %Frequency
```

```
x0 = ones(n, 1);
```

```
A0 = [1 10];
```



4.4.4. Ganho de adaptação

```
gamma = 100*eye(2*n);
```

```
tfinal = 300;
```

```
st = 0.01;
```

```
theta0 = zeros(2*n, 1);
```

```
DC = 2 %Constant
```

```
Aq = 0 %Sqr wave amplitude
```

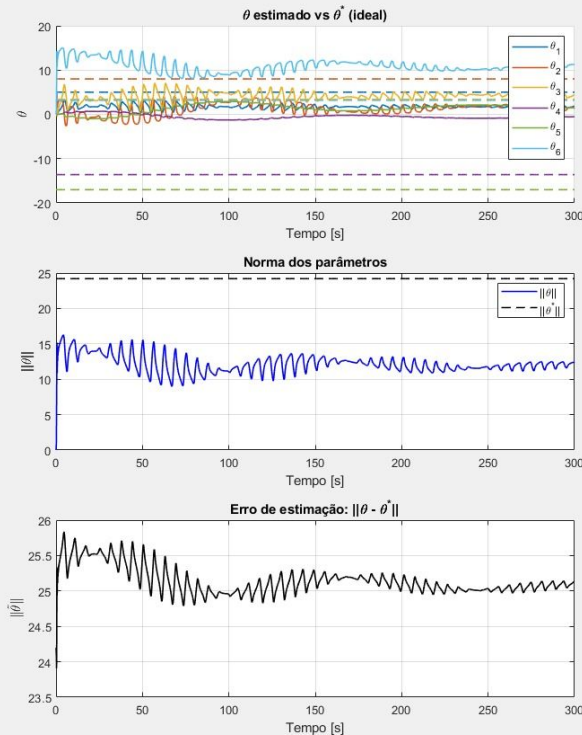
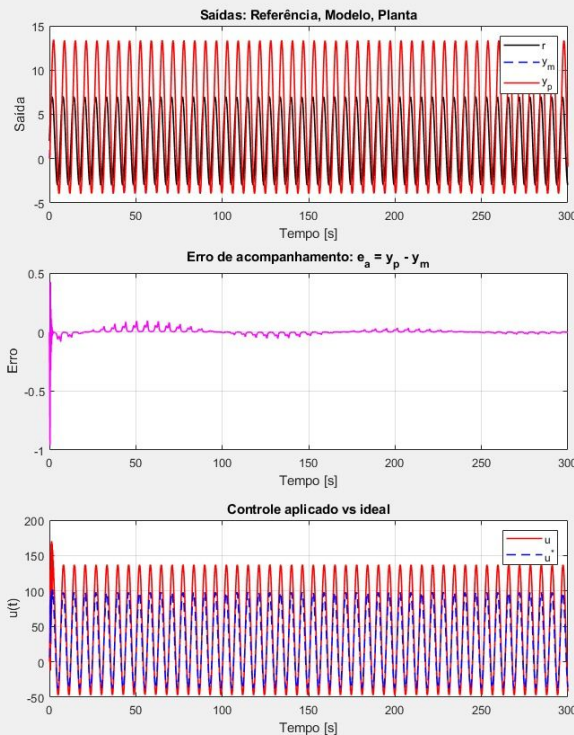
```
wq = 0.1*pi %Frequency
```

```
As = 5 %Sine wave amplitude
```

```
ws = 1%pi %Frequency
```

```
x0 = zeros(n, 1);
```

```
A0 = [1 10];
```



4.4.5. Sinal de Referência

```
gamma = 10*eye(2*n);  
tfinal = 300;  
st = 0.01;  
theta0 = zeros(2*n, 1);
```

```
DC = 2 %Constant
```

```
Aq = 2 %Sqr wave amplitude
```

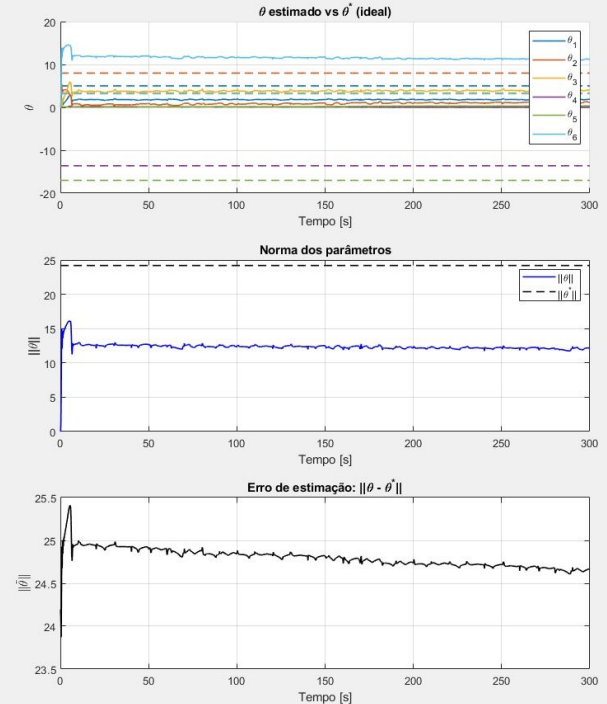
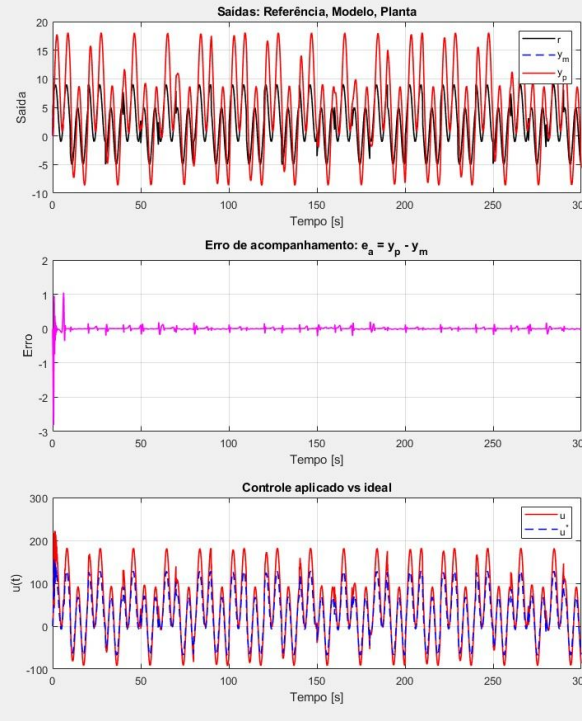
```
wq = 0.1*pi %Frequency
```

```
As = 5 %Sine wave amplitude
```

```
ws = 1%pi %Frequency
```

```
x0 = zeros(n, 1);
```

```
A0 = [1 10];
```



4.4.5. Sinal de Referência com $0.95 \theta^*$

```
gamma = 10*eye(2*n);  
tfinal = 300;  
st = 0.01;  
theta0 = 0.95*theta_star;
```

```
DC = 2 %Constant  
Aq = 2 %Sqr wave amplitude  
wq = 0.1*pi %Frequency  
As = 5 %Sine wave amplitude  
ws = 1%pi %Frequency
```

```
x0 = zeros(n, 1);
```

```
A0 = [1 10];
```

